PRACTICAL + VIVA REVISION GUIDE

Design & Analysis of Algorithms • Machine Learning • Blockchain Technology

Short practical steps, key definitions, formulas, algorithm complexities, and concise answers to the viva and assignment questions in the uploaded PDFs.

Included source files	Coverage
Assignment 1 DAA.pdf	Fibonacci series
Assignment No 2 DAA.pdf	Huffman encoding
DAA-Assignment No 3.pdf	0/1 Knapsack using dynamic programming
DAA-Assignment No 4.pdf	N-Queens using backtracking
DAA-Assignment No 5.pdf	Deterministic and randomized Quick Sort
ML_BT_ Assignment write-up(1).pdf	ML practicals 8–12 and blockchain practicals 15–19

Note: The ML/Blockchain PDF jumps from Assignment 12 to 15; Assignment numbers 13 and 14 are not present in the uploaded document. The guide follows the supplied material and does not invent those missing practicals.

Prepared as a compact exam-revision document • Use your lab manual for full program listings and screenshots.

How to use this guide

- **Practical steps:** revise the numbered workflow before explaining your code.
- **Viva definitions:** learn the bold terms and one-line meanings.
- **Formulas:** memorize the equations and know what each symbol means.
- **Complexities:** distinguish worst-case, average/expected-case, and space complexity.
- **Assignment answers:** use the short answers as a base; expand with an example if asked.

Practical index

Area	Practical / experiment	Main concept
DAA 1	Fibonacci series	Recursion, iteration, recurrence
DAA 2	Huffman encoding	Greedy algorithm, min-heap, prefix code
DAA 3	0/1 Knapsack	Dynamic programming
DAA 4	N-Queens	Backtracking
DAA 5	Quick Sort variants	Deterministic and randomized algorithms
ML 8	Uber fare prediction	Preprocessing, regression, Random Forest
ML 9	Email spam detection	Binary classification, KNN, SVM
ML 10	Bank customer churn	Artificial Neural Network
ML 11	Diabetes prediction	KNN, cross-validation, GridSearchCV
ML 12	Customer segmentation	K-Means, hierarchical clustering, elbow method
Blockchain 15–16	MetaMask and crypto wallet	Blockchain, Ether, wallet transactions
Blockchain 17	Bank smart contract	Solidity, mapping, deposit/withdraw
Blockchain 18	Student data smart contract	Struct, array, fallback, gas
Blockchain 19	Blockchain survey	Types and real-world use cases

DAA Practical 1 — Fibonacci Series

Aim: Write recursive and non-recursive programs to generate Fibonacci numbers and analyze time and space complexity.

Short theory / definitions

Fibonacci series: A sequence in which each term is the sum of the previous two terms: 0, 1, 1, 2, 3, 5, 8, 13, …

Recursion: A function calls itself until a base condition is reached. **Iteration:** Repeats statements using a loop. **Golden ratio:** The ratio of consecutive Fibonacci numbers approaches approximately 1.618.

Practical steps

1. Take n, the number of terms, as input.
2. For the iterative version, initialize a = 0 and b = 1; repeatedly calculate c = a + b and update a and b.
3. For the recursive version, use base cases F(0)=0 and F(1)=1; otherwise return F(n–1)+F(n–2).
4. Display the required terms and compare execution time / complexity.

Pseudocode

```
ITERATIVE(n):
a = 0; b = 1
if n >= 1: print a
if n >= 2: print b
for i = 3 to n:
c = a + b
print c
a = b; b = c

RECURSIVE(n):
if n <= 1: return n
return RECURSIVE(n-1) + RECURSIVE(n-2)
```

Complexity

Method	Time	Auxiliary space
Naive recursion	O(2■) (often bounded more tightly by Θ(ϕ■))	O(n) call stack
DP tabulation with array	O(n)	O(n)
Space-optimized iteration	O(n)	O(1)
Fast doubling / matrix exponentiation	O(log n)	O(log n) for recursive implementation; can be optimized

Viva / assignment answers (short)

- **How does Fibonacci work?** Start with 0 and 1; add the previous two terms to obtain the next term.
- **Fibonacci search:** Searches a sorted array by dividing its search range using Fibonacci numbers.
- **Applications:** number patterns in nature, algorithm analysis, Fibonacci search, and financial technical analysis.
- **Recurrence:** F(n)=F(n–1)+F(n–2), with F(0)=0 and F(1)=1.
- **Why is iteration better than naive recursion?** It avoids recalculating the same terms and reduces time from exponential to linear; keeping only two previous values uses O(1) space.
- **Optimization methods:** memoization, tabulation, space-optimized iteration, fast doubling, and matrix exponentiation.

DAA Practical 2 — Huffman Encoding

Aim: Implement Huffman encoding using a greedy strategy.

Short theory / definitions

Greedy method: At each step, choose the locally best available option. **Huffman coding:** A lossless compression technique that assigns variable-length prefix codes according to symbol frequency. **Prefix code:** No symbol's code is the prefix of another symbol's code. **Min-heap:** A priority queue that returns the smallest-frequency node first.

Practical steps

1. Count the frequency of every distinct character.
2. Create one leaf node per character and insert all nodes into a min-priority queue.
3. Remove the two nodes with the smallest frequencies.
4. Create a parent node with frequency equal to their sum; attach the removed nodes as children and insert the parent.
5. Repeat until one node remains; it is the Huffman tree root.
6. Traverse the tree: assign 0 to a left edge and 1 to a right edge. The root-to-leaf path is the character code.

Pseudocode

```
HUFFMAN(symbols):
Q = min_priority_queue(symbols by frequency)
while size(Q) > 1:
x = extract_min(Q)
y = extract_min(Q)
z = new_node(freq = x.freq + y.freq)
z.left = x; z.right = y
insert(Q, z)
return extract_min(Q) // root

PRINT_CODES(node, prefix):
if node is a leaf: print(node.symbol, prefix)
else:
PRINT_CODES(node.left, prefix + '0')
PRINT_CODES(node.right, prefix + '1')
```

Complexity and bit formulas

Time: O(k log k), where k is the number of distinct symbols, using a min-heap. **Space:** O(k) for the tree and queue. Frequency counting additionally takes O(n) for a message of length n.

Total encoded bits: Σ(f■ × l■). **Average code length:** Σ(f■ × l■) / N, where f■ is frequency, l■ is code length, and N is total symbols.

Worked answers

MISSISSIPPI: Frequencies: M=1, I=4, S=4, P=2. One valid set of code lengths is M=3, I=2, S=1, P=3. Total = 1×3 + 4×2 + 4×1 + 2×3 = **21 bits** (excluding tree/header storage).

Given text: ABCCDEBABFFBACBEBDFAAAABCDEEDCCBFEBFCAEFBACBEBDFAAAABC. Length = 54; frequencies A=13, B=13, C=9, D=5, E=7, F=7. One valid code assignment: A=01, B=10, C=111, D=000, E=001, F=110. Total = 136 bits; average = 136/54 ≈ 2.52 bits/character.

Viva / assignment answers

- **What tree?** A binary Huffman tree; leaves represent symbols and internal nodes represent combined frequencies.
- **Why lossless?** Prefix codes allow unambiguous decoding, so the original data can be reconstructed exactly.
- **Purpose:** Reduce the number of bits needed to represent a message by giving frequent symbols shorter codes.
- **Greedy choice:** Repeatedly combine the two least-frequent nodes.
- **How many bits?** Multiply each symbol's frequency by its code length and add the results.

DAA Practical 3 — 0/1 Knapsack using Dynamic Programming

Aim: Find the maximum profit for indivisible items subject to a knapsack capacity.

Definitions

Dynamic Programming (DP): Solves overlapping subproblems once, stores their results, and reuses them. Its key properties are **overlapping subproblems** and **optimal substructure**. **0/1 Knapsack:** Each item is either selected completely (1) or rejected (0); fractions are not allowed.

Practical steps

1. Read item weights, profits, number of items n, and capacity W.
2. Create DP table with (n+1) rows and (W+1) columns.
3. Initialize row 0 and column 0 to zero.
4. For each item and capacity, choose the better of excluding the item or including it if it fits.
5. The last cell gives maximum profit; trace upward to identify selected items.

Recurrence

Let DP[i][w] be the best profit using the first i items with capacity w:

If weight■ > w: DP[i][w] = DP[i–1][w]. Otherwise:
DP[i][w] = max(DP[i–1][w], profit■ + DP[i–1][w–weight■])

Given assignment — solved

Weights = {3, 4, 6, 5}; profits = {2, 3, 1, 4}; W=8; n=4.

Items / capacity	0	1	2	3	4	5	6	7	8
0 items	0	0	0	0	0	0	0	0	0
Item 1	0	0	0	2	2	2	2	2	2
Item 2	0	0	0	2	3	3	3	5	5
Item 3	0	0	0	2	3	3	3	5	5
Item 4	0	0	0	2	3	4	4	5	6

Optimal selection: Item 1 (weight 3, profit 2) + Item 4 (weight 5, profit 4). Total weight = 8; maximum profit = **6**.

Complexity and viva / assignment answers

DP time: O(nW), since each of n×W table entries takes constant time. **DP space:** O(nW); a 1D DP implementation can reduce space to O(W). This is pseudo-polynomial, not polynomial in the bit-length of W.

- **Greedy for 0/1?** Sorting by profit/weight ratio is a heuristic and does not always give the optimal 0/1 solution. Greedy is optimal for the fractional knapsack variant.
- **DP vs Branch and Bound:** DP stores and reuses subproblem results; Branch and Bound explores a search tree and prunes branches using bounds. DP is often preferable when W is moderate; Branch and Bound can help when bounds prune strongly.
- **NP classification:** The decision version of 0/1 Knapsack is NP-Complete; the optimization version is NP-Hard. A proposed selection can be verified in polynomial time.
- **Example:** If capacity is 5 and items are (2,10), (3,20), (4,30), choose item 1 and 2 for weight 5 and profit 30.

DAA Practical 4 — N-Queens using Backtracking

Aim: Place N queens on an N×N board, with the first queen fixed at a specified position, so that no two queens attack each other.

Definition

Backtracking: Builds a solution step by step, checks each choice, and abandons a choice as soon as it cannot lead to a valid solution. In N-Queens, no two queens may share a row, column, or diagonal.

Practical steps

1. Create an N×N board and place the first queen at the required fixed position.
2. Move to the next row and test each column.
3. A position is safe if no earlier queen is in the same column or either diagonal.
4. If safe, place the queen and recursively solve the next row.
5. If no column works, remove the previous queen and try its next possible column.
6. Stop when all N queens are placed or all possibilities are exhausted.

Pseudocode

```
SOLVE(row):
if row == N: return true
for col = 0 to N-1:
if SAFE(row, col):
board[row][col] = Q
if SOLVE(row + 1): return true
board[row][col] = '.' // backtrack
return false

SAFE(row, col):
check earlier rows for same column
check upper-left diagonal
check upper-right diagonal
return no conflict
```

Example from the supplied PDF

```
Q . . .
. . Q .
. . . Q
. Q . .
```

For N=4 with Queen 1 at row 1, column 1: Queen 2 → (2,3), Queen 3 → (3,4), Queen 4 → (4,2). No two queens share a row, column, or diagonal.

Complexity and viva

Time: O(N!) upper bound for the standard row-by-row backtracking search (the exact explored work depends on pruning and fixed placements). **Auxiliary space:** O(N) for recursion/column tracking; the board itself uses $O(N^2)$.

- **Why backtracking?** It rejects unsafe partial placements early rather than checking every complete board.
- **Applications:** Sudoku, maze solving, graph coloring, and subset-sum.
- **Safety checks:** row, column, and both diagonals.

DAA Practical 5 — Deterministic and Randomized Quick Sort

Aim: Analyze deterministic and randomized variants of Quick Sort.

Definitions

Deterministic Quick Sort: Chooses a pivot using a fixed rule, such as first, last, or middle element. **Randomized Quick Sort:** Chooses the pivot randomly. **Randomized algorithm:** Uses random choices during execution. **Las Vegas:** Always returns a correct result, but running time is random. **Monte Carlo:** Has a bounded/deterministic running time but may return an incorrect result with some probability.

Practical steps

1. Take the input array.
2. For deterministic Quick Sort, select a fixed pivot.
3. For randomized Quick Sort, choose a random index between low and high and swap it with the pivot position.
4. Partition the array into elements on either side of the pivot.
5. Recursively sort the left and right partitions.
6. Compare execution time for different input orders and sizes.

Randomized Quick Sort pseudocode

```
RANDOMIZED_QUICKSORT(A, low, high):
if low < high:
r = RANDOM(low, high)
swap A[r], A[high]
p = PARTITION(A, low, high)
RANDOMIZED_QUICKSORT(A, low, p-1)
RANDOMIZED_QUICKSORT(A, p+1, high)
```

Complexity

Case	Time	Reason
Best	O(n log n)	Partitions are balanced.
Average / expected	O(n log n)	Random pivots usually avoid repeated extreme splits.
Worst	$O(n^2)$	Every pivot is smallest/largest, giving 0 and n–1 splits.

Space: O(log n) average recursion stack; O(n) worst case. **Stability:** Standard in-place Quick Sort is not stable because swaps can change the relative order of equal keys.

Viva / assignment answers

- **Why randomize?** It makes pivot choice less dependent on input order and improves expected performance; it does not remove the $O(n^2)$ worst case.
- **Advantages of randomized algorithms:** reduce adversarial/input-order dependence, often simplify design, and provide good expected performance.
- **Analysis:** model random choices as random variables, determine outcome probabilities, and compute expected running time E[T]=ΣP■T■.
- **Quick Sort or Merge Sort?** Quick Sort often uses less auxiliary array memory and has good cache behavior; Merge Sort is stable and guarantees O(n log n) worst-case time, but usually needs O(n) extra array space.
- **Overall worst case:** $O(n^2)$, even for randomized Quick Sort.

Machine Learning — Common workflow and essential definitions

Typical supervised-learning workflow: load data → inspect/clean → separate features X and target y → encode categorical columns → split train/test → scale if needed → train model → predict → evaluate.

Core definitions

- **Data preprocessing:** Cleaning and transforming raw data into a usable form; includes missing values, duplicates, outliers, encoding, splitting, and scaling.
- **Outlier:** An observation unusually far from the general pattern. Types in the manual: global, collective, and contextual outliers.
- **Linear Regression:** Supervised method that models a continuous target as a linear function of input features.
- **Random Forest:** Ensemble of decision trees; averages tree outputs for regression and uses majority voting for classification.
- **Pandas:** Python library for tabular data using Series and DataFrame.
- **NumPy:** Python library for numerical arrays, vectorized operations, and mathematical functions.
- **Classification:** Supervised learning that predicts a category/label; binary classification has two classes.
- **Train set:** Data used to fit a model. **Test set:** Held-out data used to evaluate it.
- **random_state:** Seed used to make random splitting reproducible. **test_size=0.2:** 20% test data and 80% training data.
- **Correlation:** Measures association between variables; it does not by itself prove causation.

Scaling formulas

Min-max normalization: $x' = (x - x_{min}) / (x_{max} - x_{min})$. Usually maps values to [0,1].

Standardization: $z = (x - \mu) / \sigma$. Transforms a feature to mean 0 and standard deviation 1 (when using population σ).

Outlier IQR rule: IQR = Q3 – Q1; potential outliers are below Q1 – 1.5×IQR or above Q3 + 1.5×IQR.

Confusion matrix and evaluation formulas

Actual / predicted	Positive prediction	Negative prediction
Actual positive	TP — true positive	FN — false negative
Actual negative	FP — false positive	TN — true negative

Accuracy = (TP+TN)/(TP+TN+FP+FN). **Error / misclassification rate** = (FP+FN)/Total = 1–Accuracy.

Precision = TP/(TP+FP). **Recall / sensitivity** = TP/(TP+FN). **F1** = 2×Precision×Recall/(Precision+Recall).

Specificity = TN/(TN+FP). **FPR** = FP/(FP+TN). **ROC curve** plots true-positive rate against false-positive rate across thresholds.

Worked confusion-matrix example

	Predicted positive	Predicted negative
Actual positive	TP=40	FN=10
Actual negative	FP=5	TN=45

Total = 100. Accuracy = (40+45)/100 = **85%**. Error rate = (5+10)/100 = **15%**. Precision = 40/(40+5) = **88.89%**. Recall = 40/(40+10) = **80%**.

ML Practical 8 — Uber Fare Prediction

Objective: Preprocess the Uber fares dataset, identify outliers, check correlation, implement Linear Regression and Random Forest Regression, and compare R^2 / RMSE.

Practical steps

1. Import pandas, NumPy, Matplotlib/Seaborn and scikit-learn modules.
2. Load the Uber CSV; inspect head, shape, data types, missing values, and duplicates.
3. Clean invalid fare/coordinate records and handle missing values and outliers.
4. Create useful features such as pickup hour/day and calculate pickup-to-dropoff distance using Haversine.
5. Explore numeric columns, correlations, and box plots / heatmaps.
6. Separate X features and y fare_amount; split into train and test sets.
7. Fit LinearRegression and RandomForestRegressor.
8. Predict on test data and compare R^2, MSE, RMSE (and MAE if required).

Important formulas / definitions

Linear regression: ■ = b■ + b■x (multiple features: ■=b■+Σb■x■).

Haversine distance: a = sin²(Δϕ/2)+cosϕ■cosϕ■sin²(Δλ/2); c=2 atan2(√a,√(1–a)); d=R×c. Latitudes/longitudes must be in radians; R is Earth's radius.

MSE = (1/n)Σ(y■–■■)². **RMSE** = √MSE. **MAE**=(1/n)Σ|y■–■■|. **R^2**=1–Σ(y■–■■)²/Σ(y■–■)². Higher R^2 and lower error are generally preferred.

Viva / assignment answers

- **Why preprocess?** Real data contains missing values, noise, invalid formats and outliers; cleaning makes it suitable for modelling.
- **Outlier:** A value that deviates strongly from the rest; inspect context before removing it.
- **Random Forest regression:** Trains many decision trees on varied samples/features and averages their numerical predictions.
- **Box plot:** Displays median, quartiles, spread and potential outliers.
- **Matplotlib:** Python plotting/visualization library.

ML Practical 9 — Email Spam Detection using KNN and SVM

Objective: Classify email as Normal / Not Spam (0) or Abnormal / Spam (1), using K-Nearest Neighbors and Support Vector Machine.

Practical steps

1. Load the email CSV; inspect rows, columns, labels and missing values.
2. Separate email-word features X from spam label y; remove non-feature identifier columns if appropriate.
3. Split into training and test sets; preserve class proportions with stratify=y when suitable.
4. Scale features, especially for KNN and SVM; fit scaler on training data only, then transform test data.
5. Train KNeighborsClassifier and SVC on the training set.
6. Predict test labels; compute confusion matrix, accuracy, precision, recall and F1.
7. Compare models and tune parameters if required.

Definitions / complexity

KNN: A lazy, instance-based classifier; assigns a label by majority vote among the k nearest training examples. **Distance:** Euclidean d(x,y)=√Σ(x■–y■)².

SVM: Finds a decision boundary (hyperplane) that maximizes the margin between classes; kernels can model nonlinear boundaries.

KNN complexity: Training is typically O(1) beyond storing data; prediction by brute-force search is O(n_train×d) per query and storage O(n_train×d). **SVM:** Training complexity depends strongly on solver/kernel and data; kernel SVM can require between roughly O(n²) and O(n³) time and O(n²) memory in common formulations; linear solvers can scale better.

Viva answers

- **Binary classification:** Classification with exactly two possible labels.
- **Why scale?** KNN uses distances and SVM uses margins; large-scale features can dominate without scaling.
- **Train/test split:** Train data teaches the model; test data estimates performance on unseen examples.
- **Spam example:** Words/counts form features; spam=1 and not spam=0 are target labels.

ML Practical 10 — Bank Customer Churn using ANN

Objective: Build a neural-network classifier to predict whether a bank customer will leave within the next six months.

Practical steps

1. Read the bank customer dataset and inspect columns / missing values.
2. Choose the churn column as target y; remove identifiers and separate input features X.
3. Encode categorical features such as Geography and Gender.
4. Split into train and test sets.
5. Normalize/scale features; fit the scaler on training data and apply it to test data.
6. Build an ANN with input layer, one or more hidden layers, and output layer.
7. Compile with suitable optimizer and binary classification loss; train using training data.
8. Predict test labels and print accuracy and confusion matrix; improve model settings if needed.

Definitions / formulas

ANN: Network of connected artificial neurons/nodes arranged in input, hidden and output layers. Biological analogy in the PDF: dendrites→inputs, cell nucleus→nodes, synapse→weights, axon→output.

Activation function: Adds non-linearity. **Epoch:** One full pass through training data. **Batch size:** Number of samples processed before updating weights. **Keras:** High-level neural-network API; **TensorFlow:** ML framework.

Normalization: Often min-max scaling $x' = (x - \min)/(\max - \min)$. Use the confusion-matrix formulas in the common ML section.

Complexity

ANN training: Roughly $O(E \times n \times P)$ for E epochs, n training examples and P model parameters/operations per example; exact cost depends on layers, batch size and architecture. Model memory is $O(P)$ plus activations/batches.

ML Practical 11 — Diabetes Prediction using KNN

Objective: Predict diabetes outcome from medical predictor variables and compute confusion matrix, accuracy, error rate, precision and recall.

Practical steps from the PDF

1. Read diabetes_data.csv using pandas; inspect shape and head (the manual describes 768 rows and 9 columns).
2. Separate predictors X from the diabetes target y.
3. Use train_test_split with test_size=0.2, random_state=1 and stratify=y.
4. Create KNeighborsClassifier(n_neighbors=3) and fit it on X_train, y_train.
5. Predict X_test and evaluate accuracy / confusion matrix.
6. Use 5-fold cross-validation with cross_val_score to estimate performance across splits.
7. Tune n_neighbors with GridSearchCV over values 1 to 24 and cv=5; inspect best_params_ and best_score_.

Viva answers

- **What is K?** Number of nearest labelled samples considered to classify a new point.
- **Why odd K?** In two-class problems, an odd K can reduce tied votes (but does not guarantee no tie in multiclass settings).
- **Cross-validation:** Split data into k folds; train and validate k times, each fold used once for validation.
- **GridSearchCV:** Tests parameter combinations with cross-validation and reports the best setting.
- **random_state=1:** Makes the random train/test split reproducible; **stratify=y** preserves class proportions.

Complexity: Brute-force KNN prediction O(n_train×d) per sample; for m test samples O(m×n_train×d). Storage O(n_train×d). Cross-validation repeats fitting/evaluation k times; GridSearchCV multiplies this by the number of parameter settings.

ML Practical 12 — K-Means / Hierarchical Clustering

Objective: Cluster customer records in sales_data_sample / mall-customer data and choose K using the elbow method.

Definitions

Clustering: Unsupervised learning that groups similar observations. **K-Means:** Centroid-based algorithm that assigns each point to its nearest centroid and updates centroids. **WCSS / inertia:** Sum of squared distances from points to their assigned cluster centroids. **Elbow method:** Plot WCSS against K and select the bend where additional clusters give diminishing improvement.

Hierarchical clustering: Builds a hierarchy of clusters; agglomerative clustering repeatedly merges the closest clusters, often visualized using a dendrogram.

Practical steps

1. Load the customer dataset and inspect it; select numerical features such as Age, Annual Income and Spending Score.
2. Scale features where appropriate; exclude IDs and encode categorical data if used.
3. For K-Means, fit models for K=1 to 10 and record inertia/WCSS.
4. Plot K against WCSS; select the elbow (the manual example selects K=5).
5. Fit KMeans(n_clusters=5), predict cluster labels and add labels to the dataframe.
6. Plot clusters using a scatter plot; interpret customer groups.
7. For hierarchical clustering, compute linkage, draw a dendrogram and cut it at a chosen cluster level.

Formulas and complexity

Euclidean distance: $d(x,c)=\sqrt{\Sigma(x_{\blacksquare}-c_{\blacksquare})^2}$. **WCSS:** Σ over clusters j and points x in cluster j of $||x-\mu_{\blacksquare}||^2$.

K-Means: $O(n\times K\times d\times I)$, where n=points, K=clusters, d=features, I=iterations; space $O(n\times d + K\times d)$. **Hierarchical:** commonly $O(n^2)$ memory; naive agglomerative implementations can take $O(n^3)$, while optimized methods vary by linkage and data structure.

Viva answers

- **Supervised vs unsupervised:** Supervised learns from labelled examples; unsupervised finds structure in unlabelled data.
- **Why elbow?** It identifies a K after which WCSS decreases only slightly.
- **Uses:** customer segmentation, marketing, property grouping, document grouping and library management.
- **Silhouette score:** Measures how well a point fits its own cluster compared with the nearest other cluster; ranges from –1 to +1, higher is generally better.

Blockchain Practical 15 — MetaMask installation and Ether transaction

Objective: Learn blockchain concepts, install MetaMask, create a wallet and study Ether transaction fees.

Key definitions

- **Blockchain:** Distributed ledger of records grouped into blocks and linked using cryptographic hashes.
- **Decentralization:** No single entity controls the entire network. **Immutability:** Confirmed records are difficult to alter. **Consensus:** Network participants agree on valid transactions.
- **MetaMask:** Browser wallet used to manage Ethereum accounts and interact with decentralized applications.
- **Public address:** Shareable account identifier. **Private key / seed phrase:** Secret credentials that control wallet access; never share them.
- **Gas:** Unit measuring computational work on Ethereum. **Transaction fee:** Cost paid for executing a transaction.

Practical steps

1. Install the official MetaMask browser extension.
2. Create a wallet and set a strong password.
3. Write down the recovery phrase securely offline and verify it; never disclose it.
4. Open the wallet and note the public account address.
5. Use a suitable test network for lab work; obtain test Ether only from the approved faucet if required.
6. For a transfer, select Send, enter recipient address and amount, review the network fee, then confirm or reject.
7. Open Activity / transaction details and observe gas used, gas price, transaction hash and status.

Safety: Never enter the recovery phrase into a website claiming to be support, never share it with anyone, and verify network and recipient address before confirming.

Blockchain Practical 16 — Create a crypto wallet using MetaMask

Objective: Create a wallet and understand how cryptocurrency transactions are sent and received.

Short steps: Install MetaMask → create wallet/password → securely back up recovery phrase → copy public address → select correct network → receive funds to address or send to verified recipient → review fee → confirm → inspect Activity/history.

Hot wallet: Connected to the internet; convenient but exposed to online risks. **Cold wallet:** Keeps keys offline, often using hardware; reduces online exposure.

Bitcoin: First widely known cryptocurrency, launched in 2009. **Cryptocurrency:** Digital asset/payment system using cryptography and distributed ledgers. **Cryptojacking:** Unauthorized use of a device's computing resources to mine cryptocurrency.

Four common cryptocurrency scams

- **Ponzi / investment scam:** Old investors are paid using funds from new investors while fake profits are promised.
- **Phishing:** Fake websites/messages steal passwords, private keys or recovery phrases.
- **Fake ICO / token:** Fraudulent project collects funds with false promises.
- **Pump-and-dump:** Promoters inflate a coin's price and sell their holdings, leaving others with losses.

Are e-transfers safe? Generally safe when using trusted banking apps, encryption, OTP/PIN and verified recipient details. Never share OTP/UPI PIN; avoid unknown links and report suspicious transactions promptly.

Blockchain Practical 17 — Bank account smart contract

Objective: Write and deploy a Solidity smart contract on an Ethereum test network to deposit, withdraw and show customer balance.

Definitions

- **Smart contract:** Program deployed on a blockchain and executed by transactions/calls.
- **Solidity:** High-level, statically typed language for Ethereum smart contracts.
- **Mapping:** Key-value data structure; the manual uses mapping(address => uint256) to store each address's balance.
- **msg.sender:** Address that called the current function. **msg.value:** Ether amount sent with a payable call.
- **Reentrancy:** Vulnerability where an external call re-enters a contract before its state is safely updated.

Practical steps

1. Open Remix IDE and create a Solidity file.
2. Define a mapping from account address to uint256 balance.
3. Write payable deposit(amount): check msg.value equals amount, then add it to balanceOf[msg.sender].
4. Write withdraw(amount): require amount ≤ caller's balance; reduce balance first, then transfer Ether.
5. Expose balance using a public mapping or a view function.
6. Compile with the compatible compiler, connect a test wallet/test network, deploy and approve the transaction.
7. Test deposit, balance lookup, valid withdrawal and withdrawal exceeding balance.

Important viva points

Why update balance before transfer? To reduce reentrancy risk. Follow Checks–Effects–Interactions: validate conditions, update internal state, then interact externally.

Why require? It checks a condition and reverts the transaction if the condition is false. **Why payable?** It allows a function to receive Ether.

Complexity: Mapping lookup/update is O(1) average under the usual hash-table model; actual on-chain cost is measured in gas and depends on storage reads/writes and execution.

Blockchain Practical 18 — Student records using struct, array and fallback

Objective: Create and deploy a Solidity contract that stores student data and observe gas and transaction fees.

Definitions

- **struct:** User-defined type that groups related fields of different data types, e.g. id, name, age and course.
- **Array:** Collection of values of the same declared type; Student[] can store multiple student records.
- **Fallback function:** Special function called when calldata does not match another function, or when Ether is sent and no suitable receive function handles it (depending on contract version).
- **Gas used:** Computational units consumed. **Gas limit:** Maximum gas allowed for a transaction. **Gas price:** Price paid per gas unit.

Practical steps from the PDF

1. Open Remix IDE and create StudentData.sol.
2. Write a Student struct, a dynamic Student array, functions to add and retrieve student records, and a payable fallback function.
3. Compile using the appropriate Solidity compiler.
4. Select Deploy & Run Transactions and choose an injected wallet connected to an Ethereum test network.
5. Deploy the contract and approve the wallet transaction.
6. Observe deployment details: gas used, gas limit, effective gas price, transaction fee, transaction hash and contract address.
7. Call addStudent() to insert records; call getStudent() / public getter to retrieve records.
8. Record and compare gas used for deployment versus student insertion.

Formula and viva

Transaction fee = Gas used × Effective gas price. Example: 100,000 gas × 20 Gwei = 2,000,000 Gwei = **0.002 ETH**. Actual values vary by network conditions.

Why can deployment cost more? Deployment stores contract bytecode and initializes contract state; a simple data insertion generally performs less work, though actual gas depends on operations and data.

Blockchain Practical 19 — Survey of blockchain types and use cases

Objective: Identify blockchain network types and study their real-world applications.

Four types

Type	Who can participate?	Typical use / example
Public	Open to anyone; permissionless	Public cryptocurrency networks; Bitcoin, Ethereum
Private	Authorized participants; controlled by one organization	Internal auditing, asset management; Hyperledger deployments / Corda networks
Hybrid	Combines private control with selected public visibility	Use cases needing private records with publicly verifiable data
Consortium	Governed by multiple organizations	Banking, payment processing and shared supply-chain networks

Key features / properties

- **Distributed ledger:** Copies of records are maintained across network participants.
- **Cryptographic hashes:** Link blocks and help detect changes.
- **Digital signatures:** Prove that a transaction was authorized by the relevant key.
- **Consensus:** Nodes agree on which transactions/blocks are accepted.
- **Transparency / auditability:** Visibility depends on whether the network is public or permissioned.
- **Immutability / tamper evidence:** Altering accepted records is difficult and detectable, not magically impossible.

Assignment answers

- **Records stored:** Financial transactions, supply-chain events, certificates, identity proofs, land records, audit trails and smart-contract state. Sensitive personal data is often better stored off-chain with a hash/reference on-chain.
- **Bitcoin vs Ethereum:** Bitcoin primarily focuses on peer-to-peer digital money; Ethereum is a programmable blockchain supporting smart contracts and decentralized applications. BTC is Bitcoin's asset; ETH is Ethereum's native asset.
- **Merkle Tree:** A tree of hashes. Transaction hashes are paired and hashed repeatedly until one Merkle root remains; it supports efficient integrity verification.
- **Double spending:** Attempting to spend the same digital coin/value in more than one transaction. Consensus and transaction validation prevent conflicting spends from both being accepted.
- **Use cases:** Payments, supply-chain tracking, healthcare audit trails, credential verification, land registries, identity systems, insurance automation and smart contracts.
- **Benefits:** Shared audit trail, fewer intermediaries in some workflows, traceability and tamper evidence. Limitations include throughput, energy/cost depending on consensus, privacy, governance and regulatory concerns.

One-page complexity cheat sheet

Algorithm / method	Time complexity	Space complexity / note
Fibonacci — naive recursion	O(2■)	O(n) stack
Fibonacci — iterative	O(n)	O(1)
Fibonacci — memoization / tabulation	O(n)	O(n)
Fibonacci — fast doubling	O(log n)	O(log n) recursive stack
Huffman encoding	O(k log k)	O(k), k=unique symbols
0/1 Knapsack DP	O(nW)	O(nW), can optimize to O(W)
N-Queens backtracking	O(N!) upper bound	O(N) auxiliary; board $O(N^2)$
Quick Sort best / average	O(n log n)	O(log n) average stack
Quick Sort worst	$O(n^2)$	O(n) stack
Randomized Quick Sort expected	O(n log n)	O(log n) expected stack
KNN brute-force prediction	O(n_train×d) per query	O(n_train×d) stored data
K-Means	O(nK d I)	O(nd + Kd)
Hierarchical clustering	Often $O(n^2)$ memory; naive $O(n^3)$ time	Depends on linkage/implementation
Random Forest	Depends on trees, samples, features and depth	Stores trained trees
ANN	Approx. O(E×n×P) training	Model parameters + activations

Symbols: n = input size/items/samples; W = knapsack capacity; k = distinct symbols or clusters as defined; d = number of features; I = K-Means iterations; E = ANN epochs; P = model parameters/operations per example.

Common viva distinctions

Term pair	Difference
Recursion / iteration	Function calls itself / loop repeats steps.
Supervised / unsupervised	Uses labelled target / finds patterns without target labels.
Regression / classification	Predicts numeric continuous value / predicts class label.
Normalization / standardization	Min-max range scaling / mean 0 and standard deviation 1.
0/1 / fractional knapsack	Whole item only / fractions allowed.
Quick Sort / Merge Sort	Usually in-place, unstable / stable, typically O(n) extra array space.
Public / private blockchain	Open participation / restricted authorized participation.
Hot / cold wallet	Online keys / offline key storage.

Exam caution: Complexity for ML models depends on implementation, number of features, hyperparameters and hardware. Use the stated assumptions when answering. For Randomized Quick Sort, say *expected* O(n log n), but worst-case $O(n^2)$. For 0/1 Knapsack, O(nW) is pseudo-polynomial.

Source basis: the six PDFs uploaded in this conversation. The DAA practicals are drawn from the Fibonacci, Huffman, Knapsack, N-Queens and Quick Sort write-ups. The ML/Blockchain section follows the uploaded 42-page ML_BT assignment write-up (Assignments 8–12 and 15–19). Any explanatory complexity notes marked as implementation-dependent are concise standard analysis added to make viva answers precise.